

Logging, Monitoring & Troubleshooting

Kubernetes Production · Lesson 20

Built from the official Kubernetes documentation

kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Explain the **container logging architecture** and read logs with `kubectl logs`
- Investigate cluster **events** and use `kubectl describe` for root-cause analysis
- Deploy the **resource metrics pipeline** and read `kubectl top`
- Use `kubectl debug` **ephemeral containers** for distroless images
- Diagnose common **Pod failure states** and **node-level** problems

Reference: *Troubleshooting Applications* — kubernetes.io/docs/tasks/debug/debug-application

20.1 Container Logging Architecture

How Container Logs Work

Kubernetes captures logs from each container in a running Pod that writes to **standard output** and **standard error**.

- The **container runtime** handles and redirects `stdout` / `stderr`
- Its integration with the kubelet is the standardized **CRI logging format**
- On Linux, node-level container logs live in `/var/log/pods` (default)
- `kubectl logs` reads these node-level files **through the Kubernetes API**

Write application logs to `stdout` / `stderr` — *not* to files inside the container — so Kubernetes can collect them automatically.

Node-Level Logging & Rotation

The **kubelet** manages container logs at the node level:

- Keeps **one** terminated container (and its logs) after a restart
- Evicts **all** of a Pod's containers and logs when the **Pod is evicted**
- Rotates logs via kubelet config: `containerLogMaxSize` (default **10Mi**), `containerLogMaxFiles` (default **5**)

Caveat: `kubectl logs` returns only the **latest** log file. A Pod writing 40 MiB under 10 MiB rotation exposes at most **10 MiB** via the API.

Cluster-Level Logging

Kubernetes provides **no** native storage for logs — add a backend so logs survive crashes, evictions, and dead nodes.

Architecture	How it works
Node logging agent	DaemonSet tails <code>/var/log/pods</code> on every node
Sidecar agent	Logging container ships logs alongside the app
Direct exposure	App pushes logs straight to a backend

- The node-agent pattern (Fluentd / Fluent Bit) is the most common.

20.2 Reading Logs

kubectl logs Essentials

```
# Logs of a Pod's (single) container
kubectl logs my-pod

# A specific container in a multi-container Pod
kubectl logs my-pod -c app

# Logs from the PREVIOUS (crashed) container instance
kubectl logs my-pod --previous

# Stream/follow logs in real time
kubectl logs -f my-pod
```

`--previous` (`-p`) is the key to debugging a **CrashLoopBackOff** — it shows why the *last* container died before the restart.

Filtering Logs by Time & Label

```
# Only the last 15 minutes (also: --since-time=RFC3339)
kubectl logs my-pod --since=15m

# Last N lines, with timestamps
kubectl logs my-pod --tail=100 --timestamps

# Aggregate logs from all Pods matching a label selector
kubectl logs -l app=nginx --max-log-requests=10

# All containers of the Pod at once
kubectl logs my-pod --all-containers=true
```

- `-l` is invaluable for tailing a **Deployment's** Pods together.

20.3 Events & Describe

Cluster Events

Events record **what the control plane did** to your objects (scheduling, pulls, restarts, probe failures).

```
# Events in the current namespace, newest last
kubectl get events --sort-by=.metadata.creationTimestamp

# Events across all namespaces
kubectl get events -A

# Only events for one object
kubectl get events --field-selector involvedObject.name=my-pod
```

Events are **ephemeral** — they expire. Investigate failures promptly.

kubectl describe for Root Cause

`kubectl describe` is the single best command for *why* a Pod is unhealthy.

```
kubectl describe pod my-pod
```

It surfaces, in one view:

- **Container state**: Waiting (reason) / Running / Terminated (exit code)
- **Restart count** and last termination reason (e.g. `OOMKilled`)
- Resource requests/limits and Pod **Conditions**
- The **Events** block — usually names the exact failure

Workflow: `get pods` → `describe pod` → `logs --previous`.

20.4 The Resource Metrics Pipeline

Metrics Pipeline Components

Resource usage flows from container to CLI through four stages:

Stage	Role
cAdvisor	In-kubelet daemon; per-container CPU/memory from cgroups/CRI
kubelet	Exposes node/Pod metrics at <code>/metrics/resource</code>
metrics-server	Cluster addon; polls kubelets, serves the Metrics API
Metrics API	<code>metrics.k8s.io/v1beta1</code> ; feeds <code>top</code> , HPA, VPA, scheduler

- **metrics-server is not deployed by default** — you must install it.

kubectl top

```
# Per-node CPU & memory usage
kubectl top nodes

# Per-Pod usage (current namespace)
kubectl top pods

# Per-container breakdown, across all namespaces
kubectl top pods --containers -A
```

- `kubectl top` shows CPU in **millicores** (`m`); the raw Metrics API uses **nanocores** (`n`). Memory in **bytes** (`Ki`).
- Each value is a **window** (~30s) average — not a historical graph.

If `kubectl top` errors with *"Metrics API not available"*, metrics-server is missing or unhealthy — check `kubectl get pods -n kube-system`.

20.5 Debugging Running Pods

Exec & Ephemeral Debug Containers

```
# Get an interactive shell (image must contain a shell)
kubectl exec -it my-pod -- /bin/sh

# Attach an ephemeral debug container to a running Pod
kubectl debug -it my-pod --image=busybox --target=app
```

- `kubectl exec` works **only** if the image ships a shell.
- `kubectl debug` injects an **ephemeral container** without restarting the Pod — perfect for **distroless** images with no shell, `ps`, or package manager.
- `--target` shares the process namespace with the named container.

Debug by Copying the Pod

When you cannot disrupt the live Pod, debug a **copy**:

```
# Copy the Pod, add a debug container, share process namespace
kubectl debug my-pod -it --copy-to=my-pod-debug --share-processes

# Copy but override the command (e.g. crashloop on startup)
kubectl debug my-pod -it --copy-to=my-pod-debug -- /bin/sh

# Copy and swap a container's image
kubectl debug my-pod --copy-to=my-pod-debug --set-image=app=ubuntu
```

Ideal for a **CrashLoopBackOff** where the original command exits instantly — change the command to keep the container alive.

20.6 Troubleshooting Pod States

Common Pod Failure States

Symptom	Likely cause	First fix
Pending	No node fits; quota/taints/hostPort	<code>describe pod</code> → FailedScheduling; free resources
ImagePullBackOff	Bad image name/tag, auth, registry	Fix image; add <code>imagePullSecrets</code>
CrashLoopBackOff	App/command exits or crashes	<code>logs --previous</code> ; fix command/config
OOMKilled	Exceeded memory limit	Raise <code>limits.memory</code> or fix the leak
CreateContainerConfigError	Missing ConfigMap/Secret	Create the referenced object

Troubleshooting is **~30% of the CKA exam** — make this table reflex.

Pending & ImagePullBackOff

Pending — the scheduler could not place the Pod:

```
kubectl describe pod my-pod | grep -A5 Events  
# FailedScheduling: Node didn't have enough resource: CPU / taints / selector
```

ImagePullBackOff — the kubelet cannot pull the image:

- Check spelling and **tag** of **image:** (**:latest** may not exist)
- Private registry → ensure a valid **imagePullSecrets**
- Confirm the node has network access to the registry

CrashLoopBackOff & OOMKilled

CrashLoopBackOff — container starts then exits repeatedly:

```
kubectl logs my-pod --previous    # why the last run died
kubectl describe pod my-pod       # exit code, restart count
```

- Common causes: wrong **command/args**, missing config, failing probe.

OOMKilled — kernel killed the container for exceeding memory:

- `describe` shows `Last State: Terminated, Reason: OOMKilled`
- Increase `resources.limits.memory` or reduce the app footprint.

20.7 Node Troubleshooting

Inspecting a Node

Start from the API view, then go to the host:

```
kubectl get nodes
kubectl describe node my-node    # Conditions, capacity, allocatable
```

Watch the **node conditions**:

- **Ready=True** — healthy and accepting Pods
- **MemoryPressure** / **DiskPressure** / **PIDPressure** — resource exhaustion
- **Ready=Unknown** — kubelet stopped reporting (network/kubelet down)

A **NotReady** node with no events usually means the **kubelet** is down.

On the Node: kubelet & Runtime

The kubelet and container runtime are **not** Pods — inspect them with systemd:

```
# kubelet logs and status
journalctl -u kubelet -f
systemctl status kubelet

# containerd logs
journalctl -u containerd

# What the runtime actually sees
crictl ps -a
crictl pods
```

- For static control-plane Pods, also check `/etc/kubernetes/manifests`.

Key Takeaways

- App logs go to **stdout/stderr**; `kubectl logs` reads node files at `/var/log/pods` — use `--previous`, `-f`, `--since`, `-l`
- `describe` + `events --sort-by` pinpoint **why** an object failed
- Deploy **metrics-server** (not default) to enable `kubectl top nodes/pods`
- `kubectl debug` ephemeral containers debug **distroless** Pods
- Memorize the **Pending / ImagePull / CrashLoop / OOMKilled** table
- Node down → `journalctl -u kubelet` and `crictl` on the host

End of Lesson 20

Next: Lesson 21 — Hardening & Offline

`kubectll logs --previous` · `kubectll top pods` · `kubectll debug` · `journalctl -u kubelet`